

OpenFOAM Extensions

Walter Dal'Maz Silva

Contents

1	OpenFOAM Extensions	5
1.1	Prerequisites	5
1.2	Before starting	5
1.3	Compilation methods	5
1.4	Running tests	5
1.5	Running cases	6
2	Contributing	9
2.1	Directives	9
2.2	Wishlist	9
2.3	Building the docs	9
3	Changelog	11
3.1	2026-09-24	11
3.2	2026-09-16	11

I

Applications

4	jmakLaplacianFoam	15
----------	--------------------------	-----------

II

Libraries

5	Lagrangian	19
5.1	Parcels	19
5.1.1	Extended Thermo Parcel	19
6	Thermophysical Models	21
6.1	Thermophysical Properties	21
6.1.1	Polynomial Solid Properties	21
6.1.2	Tabulated Solid Properties	22

III

Samples

7	JMAK Solid Kinetics	27
8	Silica Particle Melting	29

IV

Tutorials

9	Tutorials	33
9.1	General	33
9.2	multicomponentFluid	33

1. OpenFOAM Extensions

Curated OpenFOAM 13 directory with additional workflow tools.

1.1 Prerequisites

This repository aims at being homogeneous in the sense it tries to keep a consistent toolset across all cases. Shell automation is done solely with bash and indirectly through Python in some cases. Unless otherwise specified, the following tools are used:

- OpenFOAM v13
- Python managed by uv
- Quarto with Typst support
- Apptainer with Docker/Podman

1.2 Before starting

Many features available in this repository rely on the environment configuration provided by `etc/bashrc`. In general the scripts will source that file for you, but environment configuration will not remain in CLI after the script finishes. To experience a smooth experience, it is recommended to source that file in your current shell from this directory:

```
source $PWD/etc/bashrc
```

If you want this to be done automatically for every new bash session, you can append that to your `~/.bashrc` by calling `foam_source_append`. This function only appends the configuration if not already present and tests for the existence of the configuration file so that your shell will not raise error messages if you delete this repository later.

1.3 Compilation methods

It is possible to build the whole extensions library or isolated features, depending on your needs. The following logic is applied throughout this sources directory:

- If a directory provides an `Allwmake` file, it allows to compile the whole tree below it; this is the case for the root directory, for instance.
- If a directory of `src/` provides a subdirectory called `Make/`, then it supports the individual library build which is done by running `wmake libso` from that directory.

All builds are written to `$FOAM_USER_APPBIN` and `$FOAM_USER_LIBBIN`.

The simplest way it to build everything from the repository root:

```
./Allwmake
```

If you encounter failures when loading a library, please consider running `./Allwclean` first to clear the build cache. Sometimes previous build artifacts may cause unexpected behavior.

1.4 Running tests

Some libraries may provide test programs under a `test/` sub-directory, and all of them are structured in the same way. The following instructions indicate how to build, run, and clean those directories.

To build and run the test suite:

```
cd test
./Allrun
```

To clean test build artifacts:

```
cd test
./Allclean
```

1.5 Running cases

Whenever possible, cases are meant to be run from within an Apptainer instance. The SIF file can be generated from the root of this repository by running `make image`. It can be later transferred to any HPC environment where Apptainer is supported, ensuring a consistent environment. Assuming you are connected to the target compute node, the following elements indicate how to instantiate an environment, run the simulation, and inspect partial results on demand. Notice that we will make use of the `screen` utility to keep services alive even if one disconnects from the node.

- Connecting to a session:

```
# Declare variables to set the case to be run:
APP_NAME="methaneAirSensitivity"
APP_PATH="tutorials/multicomponentFluid/$APP_NAME"

# A SIF generated by `make image` should be named like this:
SIF_FILE="${name}-${whoami}.sif"

# Start a named screen session:
screen -S $APP_NAME

# Start apptainer instance on the background:
apptainer instance start -B $PWD --writable-tmpfs $SIF_FILE $APP_NAME

# Enter running instance in shell mode:
apptainer shell instance://$APP_NAME
```

- From within a session, setup the base environment and run the case(s):

```
# Instantiate the environment from instance:
(cd $APP_PATH && uv sync)

# Move into the simulation directory and run:
(cd $APP_PATH && ./Allrun &)

# Ctrl + A, D to detach screen
```

- Inspecting a running simulation (from the same node as above):

```
# Attach to the running screen session
screen -r $APP_NAME

# Enter running instance in shell mode:
apptainer shell instance://$APP_NAME

# Move into the simulation directory and check status:
cd $APP_PATH

# Start Jupyter server to connect from case notebook:
uv run jupyter-notebook --no-browser --ip=0.0.0.0 \
    --ServerApp.token='' --ServerApp.password=''
```

- Stop instance after working:

```
# Stop instance after working:  
apptainer instance stop $APP_NAME
```


2. Contributing

2.1 Directives

- Comments added by this repository to explain changes performed to base libraries are tagged with `// WWW:` so that they can be tracked in the future.
- Whenever possible/applicable, copy the original `files` list from the library you are modifying to use as a starting point. Remove or comment out files that are not implemented in this repository. Modify the base path of `LIB` to use `LIB = $(FOAM_USER_LIBBIN)/<your-library-name>` so that the path is writable.
- The same apply to `options`, which requires a few extra guidelines concerning its declared variables:
 - `EXE_INC`: keep the original list untouched, add the local include directories in the end. Sometimes the library will fail to compile because the original list was lacking something (in a library as OpenFOAM it probably worked because the missing files were sourced elsewhere during compilation and the authors never had to add them), append these in the end so that we can distinguish them from the ones already in the list.
 - `LIB_LIBS`: if the project compiles but on run-time there is a missing linking library, but proceed as for `EXE_INC` by listing the library in the end. It might be tricky to identify where the functionality comes from, maybe start by `ls $FOAM_LIBBIN` and look for possibly related files.

2.2 Wishlist

- ☐ Unification/generalization of sub-classes of `solidProperties` as a new `extendedSolidProperties`; this can then be used as a replacement to `solidProperties` in the `extendedThermoParcel` framework (possible need to generalize the cloud definition).

2.3 Building the docs

A `Makefile` is provided in the root directory: running `make` will generate a local documentation, `make publish` can be used by admins to publish to GitHub Pages, and `make clean` strips the generated outputs and intermediate files from the folder.

3. Changelog

3.1 2026-09-24

- Added an exhaustive, readable, and well-commented test suite in `etc/test.sh` covering all shell functions under `etc/rc/` (`utilsrc`, `coresrc`, `meshingrc`) with strict 80-character line wrapping, deterministic mocking, and comprehensive edge-case assertions.
- Enhanced `safe_run_ncores` in `etc/rc/coresrc` to accept an arbitrary sequence of fallback core counts. If the primary requested core count exceeds physical cores, fallback values are evaluated in order; if none are feasible or provided, execution falls back to half of the available physical cores. Also changed display of messages by sourcing `utilsrc` from `corerc` and improved re-sourcing time by testing for `WM_PROJECT` in `bashrc`.

3.2 2026-09-16

- Adapted `extendedThermoCloud` to upcast the solid object to a `tabulatedSolid` as a prototype for temperature dependent properties. Currently this supports a single species and need to be generalized in the future.
- Unified the build of all libraries under a single `Allwmake` at the sources root, following the full library standard. Added compilation instructions at multiple levels to `README.md`
- Added `extendedThermoCloud` support by extending `thermoCloud`; this extension if fact only needed working with `thermoParcel` as the physics was found to be at the parcel level. Added related library sources under `lagrangian/parcel/{cloud,parcels}`.

I

Applications

4	<code>jmakLaplacianFoam</code>	15
---	--	----

4. jmakLaplacianFoam

Solver for time-temperature reactive heat release in solids. Implements differential Johnson–Mehl–Avrami–Kolmogorov (JMAK) reaction progress kinetics for non-isothermal process as proposed by [Mittemeijer et al.](#).

In the future, it would be desirable to have the following features implemented:

- ☐ Handle non-linearities in thermal properties.
- ☐ Generalization for an arbitrary number of reactions.
- ☐ Implement `solidThermo` to use `externalWallHeatFluxTemperature`.

This solver was initially implemented and tested with OpenFOAM v2212 and later migrated to OpenFOAM 13. The original implementation is made available under [legacy](#) directory and is no longer maintained.

II

Libraries

5	Lagrangian	19
5.1	Parcels	19
6	Thermophysical Models	21
6.1	Thermophysical Properties	21

5. Lagrangian

5.1 Parcels

5.1.1 Extended Thermo Parcel

Provides a variant of `thermoCloud` with varying specific heat.

Important: this extension currently support a single solid phase which needs to be a `tabulatedSolid` as provided in the thermophysical properties below. This will need to be generalized at some point.

Extending a parcel requires several templates to be adapted. The following building blocks are present in the implementation of `extendedThermoCloud`:

- Parcel template class is found in [ExtendedThermoParcel](#)
- The derived parcel instantiation [extendedThermoParcel](#)
- The derived cloud instantiation [extendedThermoCloud](#)

Or in a tree view:

```
parcel
├── clouds
│   └── derived
│       └── extendedThermoCloud
│           ├── extendedThermoCloud.C
│           └── extendedThermoCloud.H
└── parcels
    ├── Templates
    │   └── ExtendedThermoParcel
    │       ├── ExtendedThermoParcel.C
    │       ├── ExtendedThermoParcel.H
    │       ├── ExtendedThermoParcelI.H
    │       ├── ExtendedThermoParcelIO.C
    │       ├── ExtendedThermoParcelName.C
    │       └── ExtendedThermoParcelTrackingDataI.H
    └── derived
        └── extendedThermoParcel
            ├── extendedThermoParcel.H
            └── makeExtendedThermoParcelSubmodels.C
```

For using it, append the following to `controlDict` (it depends on extended thermophysical properties so that non-constant properties are supported):

```
libs
(
    "libextendedThermophysicalProperties.so"
    "libextendedLagrangianParcel.so"
);
```


6. Thermophysical Models

6.1 Thermophysical Properties

6.1.1 Polynomial Solid Properties

Library: thermophysicalProperties/solidProperties/polynomialSolidProperties

Extends OpenFOAM's `solidProperties` base class using `Polynomial<8>` to support temperature-dependent 8th-order polynomials density ($\rho(T)$), specific heat capacity ($C_p(T)$), and thermal conductivity (κ).

It is structured as follows:

```
├─ Make
│   └─ files
│       └─ options
├─ polynomialSolid.C
├─ polynomialSolid.H
├─ polynomialSolidI.H
└─ test
    ├── Allclean
    ├── Allrun
    ├── Make
    │   ├── files
    │   └─ options
    └─ testPolynomialSolid.C
```

In `system/controlDict` one must load the compiled library at startup:

```
libs
(
    "libpolynomialSolidProperties.so"
);
```

In `constant/physicalProperties` you can now specify `polynomialSolid` under `solids {}`:

```
solids
{
    polynomialSolid
    {
        molWeight    60.084;
        Hf            0.0;
        emissivity    0.9;

        // Polynomial coefficients for rho(T) = r0 + r1*T + r2*T^2 + ...
        rhoCoeffs     (2500 -0.1 0 0 0 0 0 0);

        // Polynomial coefficients for Cp(T) = c0 + c1*T + c2*T^2 + ...
        CpCoeffs       (800 0.5 -1e-4 0 0 0 0 0);

        // Polynomial coefficients for kappa(T) = c0 + c1*T + c2*T^2 + ...
        kappaCoeffs    (1.5 0 0 0 0 0 0 0);
    }
}
```

6.1.2 Tabulated Solid Properties

Extends OpenFOAM's `solidProperties` base class using `Function1<scalar>` to support tabulated temperature-dependent density ($\rho(T)$), specific heat capacity ($C_p(T)$), and thermal conductivity (κ).

It is structured as follows:

```

├─ Make
│   └─ files
│       └─ options
├─ tabulatedSolid.C
├─ tabulatedSolid.H
├─ tabulatedSolidI.H
└─ test
    ├── Allclean
    ├── Allrun
    ├── Make
    │   ├── files
    │   └─ options
    └─ testTabulatedSolid.C

```

In `system/controlDict` one must load the compiled library at startup:

```

libs
(
    "libtabulatedSolidProperties.so"
);

```

In `constant/physicalProperties` you can now specify `tabulatedSolid` under `solids {}`:

```

solids
{
    tabulatedSolid
    {
        molWeight    60.084;
        Hf            0.0;
        emissivity    0.9;

        // Tabulated density rho(T) [kg/m^3]
        rho
        table
        (
            (300 2500)
            (600 2450)
            (1200 2380)
        );

        // Tabulated specific heat Cp(T) [J/(kg K)]
        Cp
        table
        (
            (300 800)
            (500 950)
            (800 1100)
            (1200 1250)
        );

        // Tabulated thermal conductivity [W/(m K)]
        kappa
        table
        (
            (300 1.5)
            (500 1.5)

```

```
        (800 1.5)
        (1200 1.5)
    );
}
}
```


III

Samples

7	JMAK Solid Kinetics	27
8	Silica Particle Melting	29

7. JMAK Solid Kinetics

i About

Sample case can be found [here](#). It illustrates the use of `jmakLaplacianFoam` solver provided in this repository.

Simulate the temperature evolution of a reacting cylinder (diameter 3 inches and height of 6 inches) immersed in coded external convective boundary conditions representing steady air at room temperature. The solid is undergoing volumetric reactions represented by a JMAK kinetics with heat release, leading to a thermal build-up near the sample core. This could represent, *e.g.*, a cement hydration process.

Figure 7.1 illustrates the temperature evolution at a probe placed near the core of the cylinder. The humps are associated to the different reactions that take place over time.

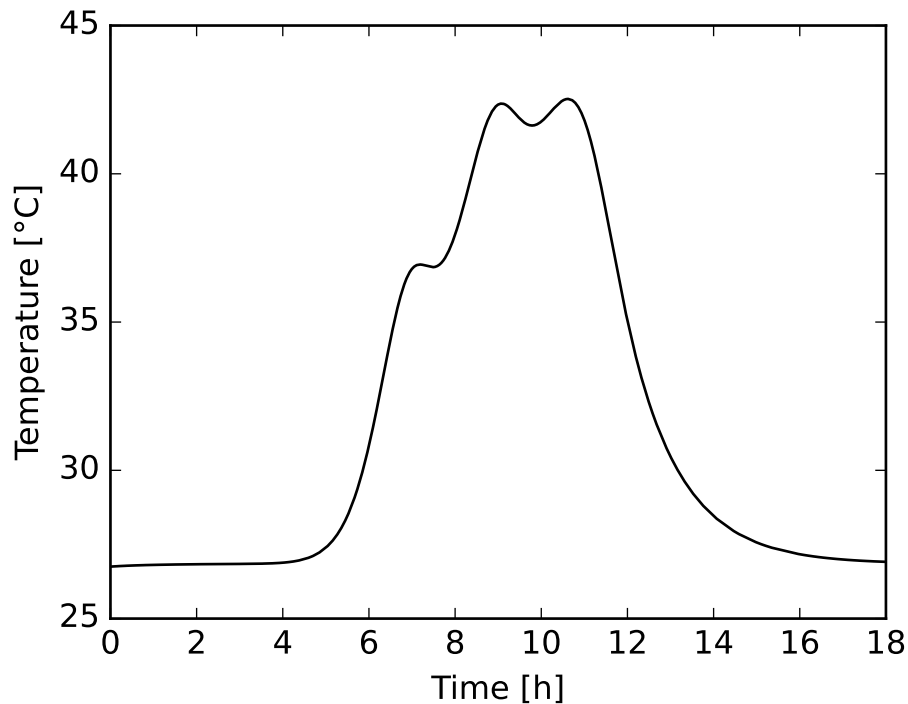


Figure 7.1: Thermal history at probe location.

Figure 7.2 demonstrates how the temperature is distributed in a vertical cut of the cylinder after 8h. Plots of *beta* indicate the JMAK progress of the reaction: we see that reaction 1 has saturated at 1.23, enabling reaction 2 that has already progressed considerably, and reaction 3 is still in earlier stage.

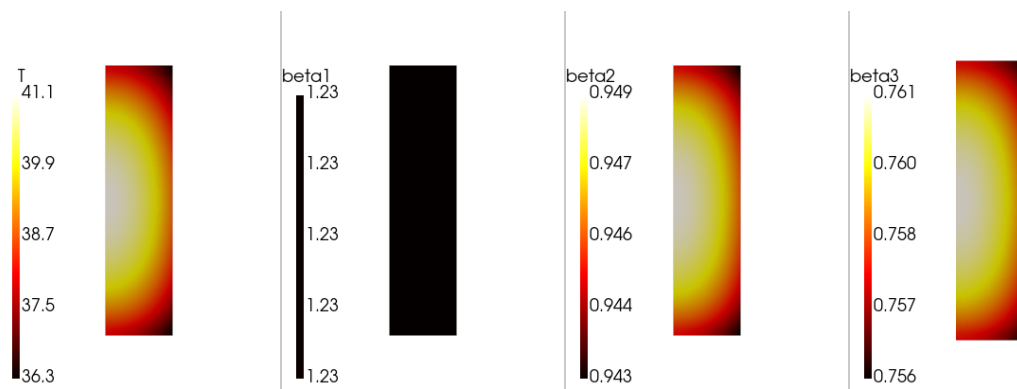


Figure 7.2: Temperature profile at a slice of the domain after 8h.

8. Silica Particle Melting

i About

Sample case can be found [here](#). It intends to illustrate the proper capture of a solid particle melting, as supported by the following libraries provided in this repository:

- `libextendedThermophysicalProperties`: provides `tabulatedSolid`, used for representation of piecewise-linear silica properties, allowing the incorporation of the *lumped* reaction enthalpies in specific heat.
- `libextendedLagrangianParcel`: provides `extendedThermoCloud` allowing the simulation of non-constant properties lagrangian particles.

Simulates the heating of a single silica particle immersed in the flue gases from propane-oxygen combustion. Fluid properties were processed with Cantera and fit to polynomials to match the temperature dependence of adiabatic combustion flue gases (equilibrium computed under constant enthalpy and pressure). Solid specific heat capacity was modified to include smeared latent heat of phase transitions where applicable.

Figure 8.1 displays the thermal history of a single particle under the above conditions. Phase changes produce flat regions, notably the large plateau observed in between 23 – 32 ms associated to the melting of the particle, thus representing the intended goal of capturing the expected silica latent heat of fusion. Since the domain has been intentionally oversized with respect to the required energy to melt the particle, we observe a temperature increase beyond melting point, as expected.

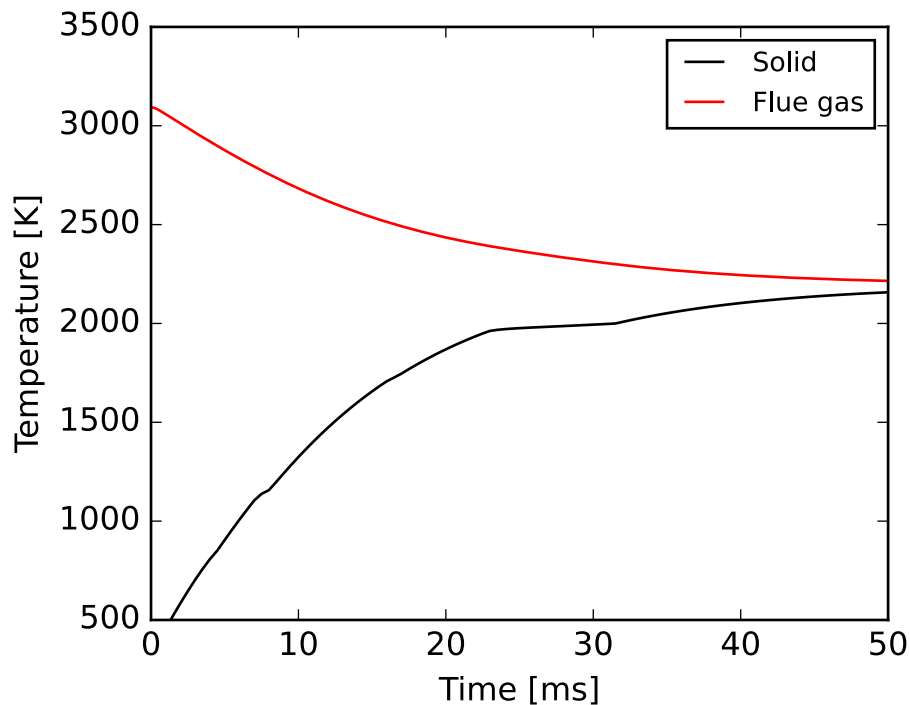


Figure 8.1: Verification of single particle heating thermal history.

i To-do

- Simulate the same case with the base `thermoCloud` to illustrate the uncontrolled heating the particle would undergo without latent heat incorporation.

IV

Tutorials

9	Tutorials	33
9.1	General	33
9.2	multicomponentFluid	33

9. Tutorials

Unlike the sample cases, which are intended to be minimal working examples of the extensions provided here, the tutorials are full fledged case studies that may or may not include the extensions provided here. These illustrate the authors main skills and document complex workflows that go beyond the scope of tutorials provided with OpenFOAM.

9.1 General

WiP

9.2 multicomponentFluid

- [methaneAirSensitivity](#): illustrates the simulation of methane combustion in a co-axial burner under different operating conditions. The main goals are to provide a robust setup for combustion simulations.